[8]

Cloning and Fetching

In the last chapter, you learned about remote repositories and you created one for the Rainbow project.
In this chapter, you are going to start simulating what it would be like to work with a friend on the Rainbow project. To do that, you are going to learn about cloning remote repositories and how this differs from initializing repositories locally. You will also learn more about defining upstream branches, deleting branches, and fetching data from a remote repository.

[NOTE]

Some text editors that have Git integrations allow you to define certain settings for Git. This chapter assumes that you have not enabled any special settings that will cause Git to deviate from its default behavior.

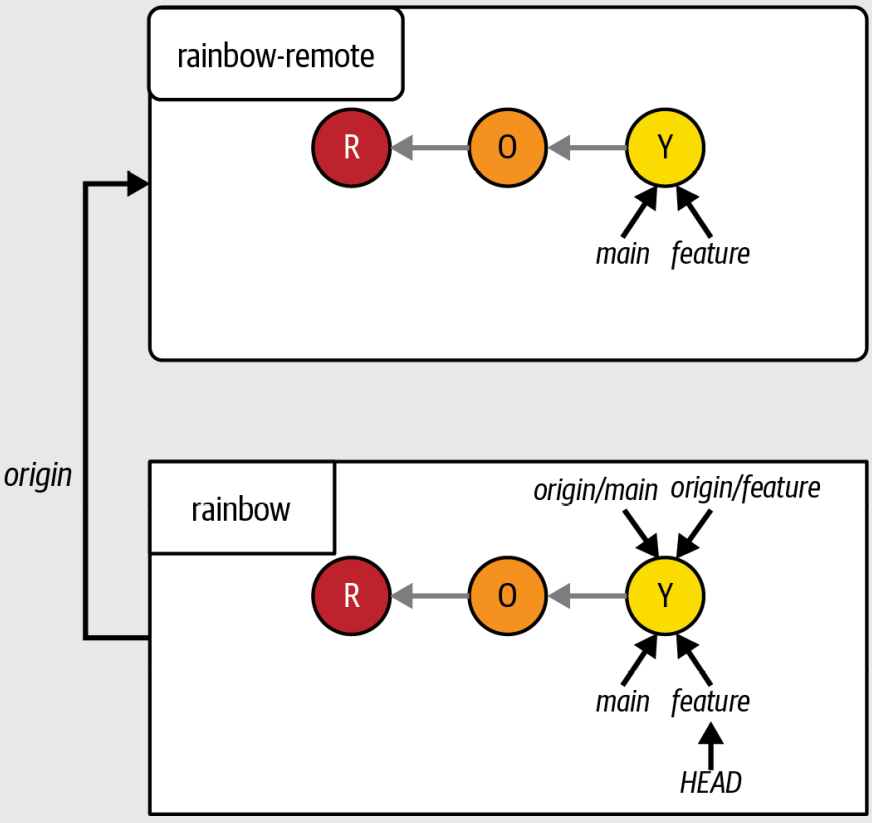
For example, Visual Studio Code has a feature called Autofetch (git.autofetch) that will periodically fetch changes from a remote repository. (You will learn about fetching later in this chapter.) This feature is disabled by default, and for you to be able to do the exercises in this chapter properly it should remain disabled. If you have enabled it in the past, then please disable it before continuing.

If you have never configured any special Git settings for your text editor, you don't need to worry about this.

State of the Local and Remote Repositories

At the start of this chapter, you should have a local repository called rainbow and a remote repository called rainbow-remote. These two repositories should be in sync; that is, they should contain the same commits and branches. The expected current state of the two repositories is illustrated in Visualize it 8-1.

[VISUALIZE IT 8-1]



The Rainbow project at the start of Chapter 8, with a local repository called rainbow and a remote repository called rainbow-remote

Cloning a Remote Repository

In the previous chapter you created a remote repository on a hosting service so you could show your friend the work you've been doing on the Rainbow project. Now, let's pretend that your friend has decided they want to help you work on the project. If they're going to contribute to the project, they will need to clone (that is, copy) your re-

mote repository. In the following section I will describe how you will simulate this scenario.

At the start of the book, you learned that Git is a useful tool for collaboration. Cloning remote repositories is an essential part of being able to collaborate with other people on a Git project, since it allows them to work with their own copy of the repository on their computer. An unlimited number of people can clone a remote repository (as long as they are provided with access to it).

Let's take a look at Example Book Project 8-1 for an illustration.

EXAMPLE BOOK PROJECT 8-1

Suppose I initially planned to work on my Book project on my own, then later changed my mind and decided that I wanted to work with a coauthor.

My coauthor will need to clone the remote repository to have their own copy of the Book project on their computer so they can start working with me on the book. To enable this, they will need to have an account with whichever hosting service is hosting the remote repository. Then, I will have to grant them editing access to the repository so they can comment and contribute, regardless of whether it is public or private.

If at some point I want my editor to review my work, we'll need to follow the same steps: they'll set up an account with the hosting service, then I'll grant them access to the repository and tell them where to find it, and they will clone the repository onto their computer.

Example Book Project 8-1 shows why cloning repositories is such an important part of collaborating with others on any Git project. Next, we'll look at how you are going to simulate the experience of working with someone else on the Rainbow project.

THE COLLABORATION SIMULATION

Normally, if two people are working on the same project, each will have a local repository on their own computer and each will contribute to one remote repository. Given that you may not have two computers or another person available to help you work through the collaboration exercises in this and the following chapters, you are going to clone the remote repository onto your computer and create a second local repository, which you will call friend-rainbow to distinguish it from the rainbow repository.

You will then pretend that this second local repository is on your friend's computer. From now on, when we refer to your friend completing some action, you will perform the action yourself in the friend-rainbow repository.

After your friend clones the remote repository, you will be working with two local repositories. I recommend working with two separate text editor windows, one for each project directory. I also recommend working with two separate command line windows (or integrated terminals), one for each repository, rather than navigating into and out of each project directory in one command line window.

In Git, we use the term *clone* or *cloning* to refer to the process of copying a remote repository onto a computer to create a local repository and the command we use to do this is git clone. To clone a remote repository, you use the git clone command and pass in the remote repository URL and, optionally, a project directory name.

[SAVE THE COMMAND]

git clone <URL> <directory_name>

> Clone a remote repository

If you don't pass a project directory name, then the local repository will be assigned the remote repository project name. For example, if you don't pass a project directory name in the upcoming Follow Along when you clone the rainbow-remote repository, then your friend's local repository will also be called rainbow-remote. Because that would make this learning exercise very confusing, instead you are going to pass the project directory name friend-rainbow to the git clone command as an argument to create a local repository of that name.

The git clone command does the following:

1. Create a project directory inside the current directory.
2. Create (initialize) the local repository.
3. Download all the data from the remote repository.
4. Add a connection to the remote repository that was cloned; by default it will have the shortname origin in the new local repository.

To explicitly see the repository being created, you can clone it into your desktop directory. This will allow you to see the new project directory appear on the desktop of your computer.

Go to Follow Along 8-1 to clone the remote repository. In step 3, be sure to enter the entire command on a single line.

[FOLLOW ALONG 8-1]

1. Open a new command line window and navigate to the desktop directory.

2. Go to your hosting service and copy your remote repository URL for the protocol you have chosen to use, either SSH or HTTPS. You will use this URL in step 3 of this Follow Along.

3. desktop $ **git clone https://github.com/gitlearningjourney/rainbow-remote.git friend-rainbow**
   Cloning into 'friend-rainbow'...
   remote: Enumerating objects: 9, done.
   remote: Counting objects: 100% (9/9), done.
   remote: Compressing objects: 100% (4/4), done.
   remote: Total 9 (delta 1), reused 9 (delta 1), pack-reused 0
   Receiving objects: 100% (9/9), done.
   Resolving deltas: 100% (1/1), done.

4. Locate the newly created friend-rainbow project directory in your filesystem or on your desktop.

What to notice:

- You cloned the remote repository onto your computer and created a second local repository called friend-rainbow.
- You are not in the friend-rainbow directory in the command line, because cloning a Git repository does not mean you automatically navigate into it.

Go to Follow Along 8-2 to navigate into the new friend-rainbow project directory to explore its contents and setup.

[FOLLOW ALONG 8-2]

1. desktop $ **cd friend-rainbow**

2. Open the friend-rainbow project directory in a new text editor window.

3. friend-rainbow $ **git remote -v**
   origin https://github.com/gitlearningjourney/rainbow-remote.git (fetch)
   origin https://github.com/gitlearningjourney/rainbow-remote.git (push)

4. friend-rainbow $ **git branch --all**
   * main
   remotes/origin/HEAD -> origin/main
   remotes/origin/feature
   remotes/origin/main

5. friend-rainbow $ **git log**
   commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> main, origin/main, origin/feature, origin/HEAD**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 10:09:59 2022 +0100
   yellow
   commit 7acb333f08e12020efb5c6b563b285040c9dba93
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange

What to notice:

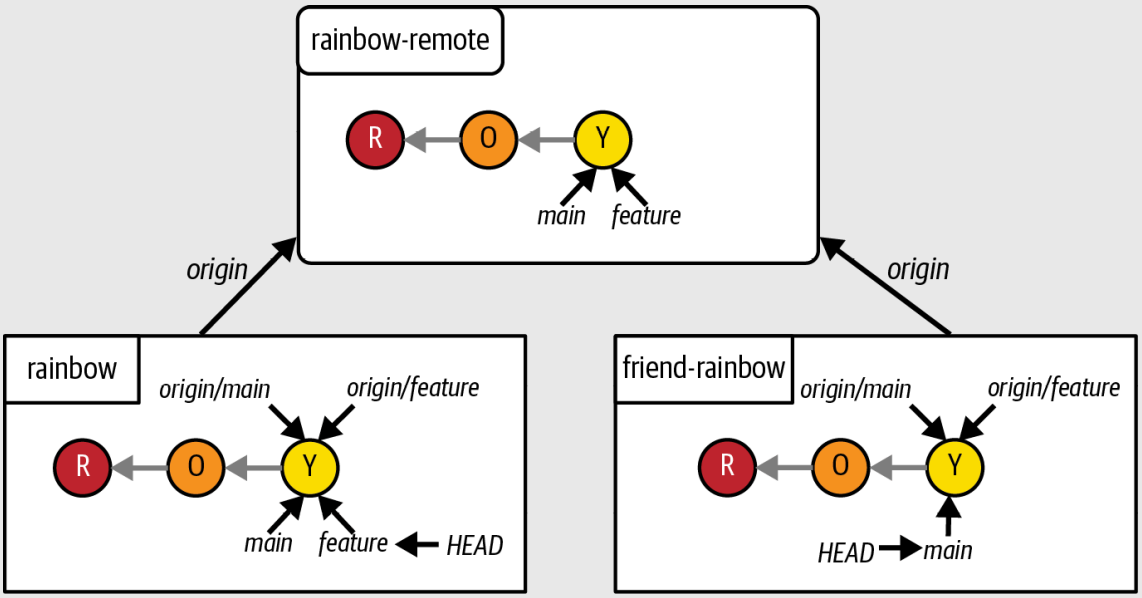
- In step 1, you used the cd command to navigate into the friend-rainbow repository in the command line. This is necessary because cloning a repository does not mean you navigate into it automatically.
- In step 3, in the git remote output, the origin remote repository shortname is already listed.
- In step 4, the git branch output shows a pointer called origin/HEAD that points to the origin/main remote-tracking branch, and that you have an origin/feature remote-tracking branch. You can also see that there is no local feature branch.

All of the preceding observations are illustrated in Visualize it 8-2.

[NOTE]

From now on, I'll expand the Repository Diagrams to depict the state of the two local repositories, rainbow and friend-rainbow, as well as the remote repository, rainbow-remote.

[VISUALIZE IT 8-2]



The Rainbow project after your friend clones the remote repository and creates a local repository called friend-rainbow

In the next sections, we will dive deeper into what happens during the cloning process and answer the following questions:

- What is the origin/HEAD pointer?
- Why is there no local feature branch in the new friend-rainbow local repository?
- Why was the origin shortname automatically created for the new local repository?

WHAT IS ORIGIN/HEAD?

Earlier, you noticed that there is a pointer called origin/HEAD in the friend-rainbow repository. What is this?

When you clone a repository, Git needs to know which branch it should be on when it's done cloning. The origin/HEAD pointer determines which branch this is. In the Rainbow project, origin/HEAD points to the main branch, which is why your friend that cloned the repository is on the main branch in the friend-rainbow repository.

> **[NOTE]**
>
> For simplicity, I will not include the origin/HEAD pointer in the Visualize It diagrams.

By contrast, notice that in the rainbow repository you are currently on the feature branch —but in the friend-rainbow repository, the local feature branch doesn't even exist. You'll learn why that is in the next section.

CLONING REPOSITORIES AND DIFFERENT TYPES OF BRANCHES

In the git log output produced by step 5 of Follow Along 8-2, you can see that in the friend-rainbow repository there is no reference to the local feature branch. However, there is a reference to the origin/feature remote-tracking branch. This is because when you clone a repository the git clone command will create remote-tracking branches for all the branches currently present in the remote repository that is being cloned, but the only local branch that is created is the branch that origin/HEAD points to.

For your friend to work on the feature branch, they must switch onto it. Then Git will create a local feature branch based on where the remote-tracking branch was pointing.

Go to Follow Along 8-3 to simulate your friend switching onto the feature branch.

[FOLLOW ALONG 8-3]

1. friend-rainbow $ **git branch --all**

```
* main
remotes/origin/HEAD -> origin/main
remotes/origin/feature
remotes/origin/main
```

2. friend-rainbow $ **git switch feature**

```
branch 'feature' set up to track 'origin/feature'.
Switched to a new branch 'feature'
```

3. friend-rainbow $ **git branch --all**

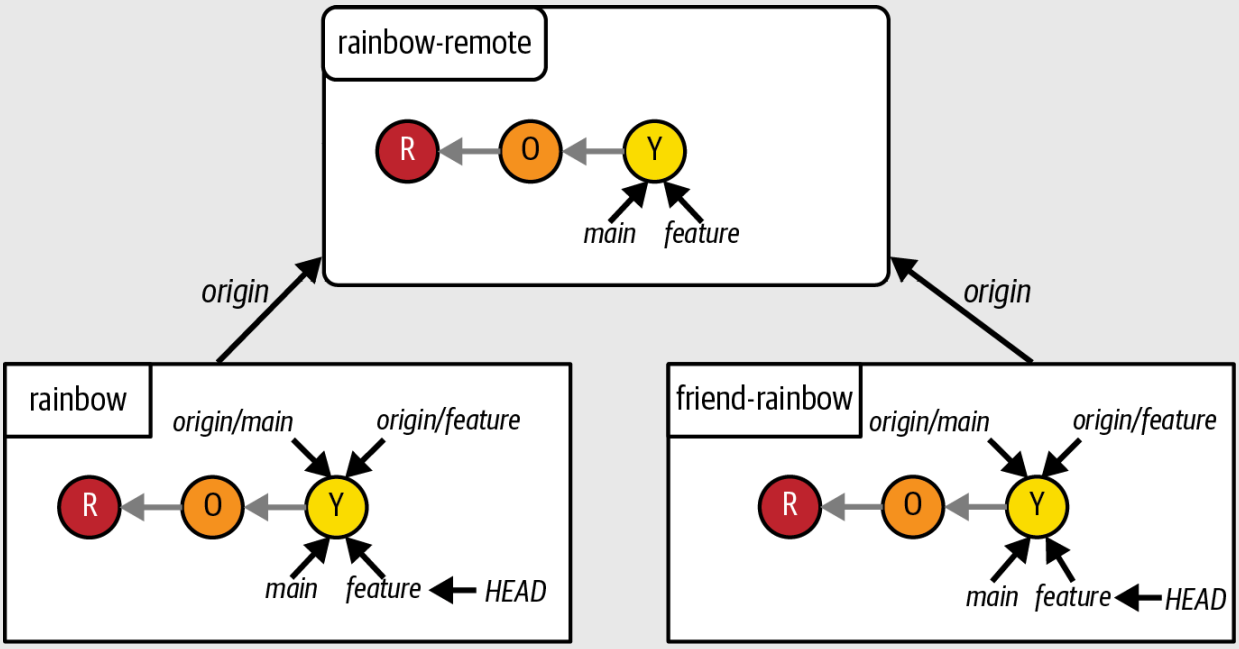
```
* feature
main
remotes/origin/HEAD -> origin/main
remotes/origin/feature
remotes/origin/main
```

What to notice:

- In step 3, in the git branch --all output, you can see that there is a new local feature branch and your friend is on it.

This is illustrated in Visualize it 8-3.

[VISUALIZE IT 8-3]



The Rainbow project after your friend switches onto the feature branch

You've just learned how to switch onto and create new local branches based off remote-tracking branches that you downloaded from a remote repository. Next, let's discuss why the friend-rainbow repository already has the origin shortname assigned to the connection to the remote repository.

THE ORIGIN SHORTNAME

Recall that in Chapter 7, you learned that for a local repository to communicate with a remote repository, the local repository must have a connection with a shortname to the remote repository stored within it. To work with the remote repository, you had to explicitly associate its URL with a shortname using the git remote add <shortname> <URL> command. This is because you created the rainbow repository locally using the git init command, and it had not yet had any interaction with the remote repository.

However, in step 3 of Follow Along 8-2, in the git remote output, you saw that you already have the origin shortname listed. This means the friend-rainbow repository already has the remote repository URL associated with the shortname origin. This is because your friend's local repository did not originate locally; it was directly cloned from a remote repository. At the time of cloning, the remote repository URL was associated with a

shortname in the local repository, and origin is the default shortname Git associates with a remote repository when you clone it.

Now that you know a bit more about the cloning process, let's talk some more about branches. In Chapter 4, you learned how to create and switch between branches. Next, you'll learn how to delete them.

Deleting Branches

The main reason to delete branches is to keep a Git project organized and uncluttered. Before deleting a branch, you should always make sure that either you have merged it into another branch or you're sure you don't want to use any of the work that is found only on that branch.

[NOTE]

When you delete a branch with commits that are not part of any other branch, you don't delete the commits that are part of that branch. The commits still exist in your commit history. However, they are no longer easy to reach, because there is no simple branch reference to them and they are not part of the development history of any existing branch.

To demonstrate how branch deletion works, let's assume that your friend decides they don't need the feature branch, so they want to remove it. To fully delete a branch, you need to delete the remote branch, the remote-tracking branch, and the local branch. To delete a remote branch and a remote-tracking branch, you use the git push <shortname> -d <branch_name> command (where -d stands for "delete"). With this command, you essentially upload a deletion to the remote repository. You must pass it the name of a remote branch.

[SAVE THE COMMAND]

***git push** <shortname> **-d** <branch_name>*

Delete a remote branch and the associated remote-tracking branch

[NOTE]

You can also delete a remote branch directly on the website of a hosting service, but keep in mind that this will not delete the remote-tracking branch. We'll walk through this process in "Deleting Remote Branches" on page 249.

To delete a local branch, you use the git branch command with the -d option, passing in the name of the branch you want to delete.

[SAVE THE COMMAND]

git branch -d <branch_name>

Delete a local branch

Go to Follow Along 8-4 to carry out the branch deletion on behalf of your friend. Note that you cannot be on a branch when you delete it, so you (acting as your friend) will have to switch from the feature branch to the main branch in the friend-rainbow repository to delete the local feature branch.

[FOLLOW ALONG 8-4]

1. friend-rainbow $ **git branch --all**
   * feature
   main
   remotes/origin/HEAD -> origin/main
   remotes/origin/feature
   remotes/origin/main

2. friend-rainbow $ **git push origin -d feature**
   To github.com:gitlearningjourney/rainbow-remote.git
   - [deleted] feature

3. Go to the rainbow-remote repository on your hosting service, refresh the page, and look at the list of branches. The feature branch should no longer be there.

4. friend-rainbow $ **git branch --all**
   * feature
   main
   remotes/origin/HEAD -> origin/main
   remotes/origin/main

5. friend-rainbow $ **git switch main**
   Switched to branch 'main'
   Your branch is up to date with 'origin/main'.

6. friend-rainbow $ **git branch -d feature**
   Deleted branch feature (was fc8139c).

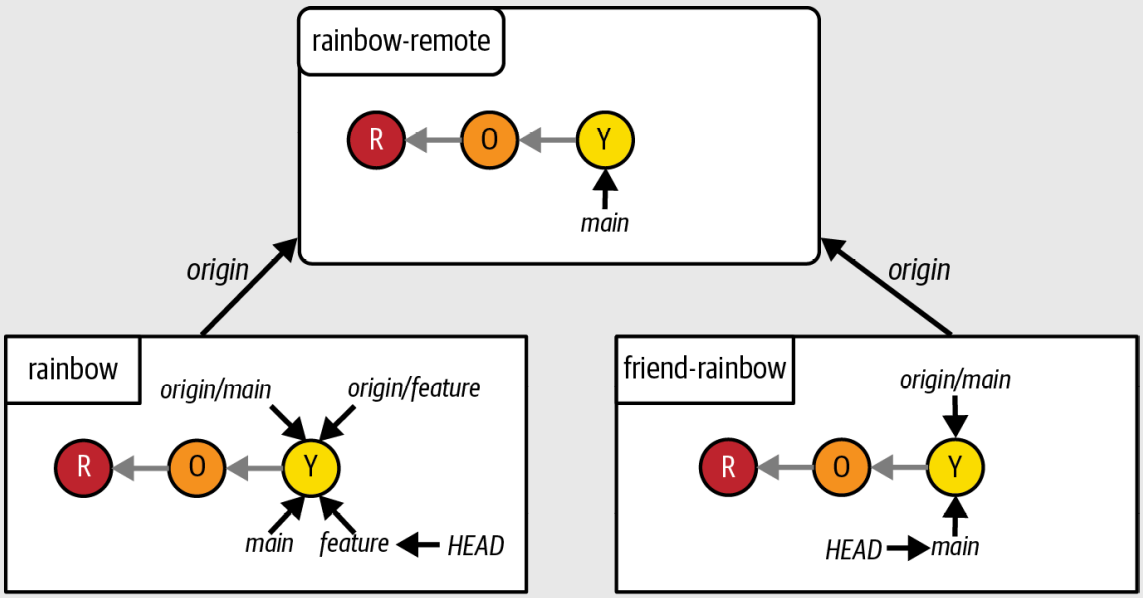
7. friend-rainbow $ **git branch --all**
   * main
   remotes/origin/HEAD -> origin/main
   remotes/origin/main

What to notice:

- In step 2, you deleted the remote feature branch and the origin/feature remote-tracking branch.
- In step 3, you can see there is no remote feature branch in the rainbow-remote repository.
- In step 6, you deleted the local feature branch.
- In step 7, the git branch --all output indicates that there is no local feature branch in the friend-rainbow repository.

Visualize it 8-4 illustrates the state of the local and remote repositories after making these changes.

[VISUALIZE IT 8-4]



The Rainbow project after your friend deletes the remote feature branch, the origin/feature remote-tracking branch, and the local feature branch

As you can see, in the rainbow repository there still is a feature branch and an origin/feature remote-tracking branch. The fact that your friend deleted the feature and origin/feature branches in their friend-rainbow repository does not affect your rainbow repository. You'll delete these branches in the rainbow repository later in this chapter, but for now, let's learn about how the collaboration works when your friend starts contributing to the Rainbow project.

Git Collaboration and Branches

Now that you're going to start collaborating with others on the Rainbow project, you may want to discuss what some of your Git conventions should be. For example, in Chapter 3 I mentioned that some teams may have rules about what to include in a commit message, and in Chapter 4 you learned that some teams may also have conventions about how they name their branches.

Teams also may have rules about how to manage branches. For example, there may be conventions about:

- Which branches are allowed to be merged into one another.
- When branches need to be created.
- What the review process for work on a branch should be.

An example of a rule that you may encounter in a Git project is that individuals should work only on their own topic branches and avoid working on other people's topic branches. This helps avoid merge conflicts (which we will cover in Chapter 10). We won't apply this rule to the Rainbow project in this book, to keep things simple and allow you to focus on the new concepts you are learning in each chapter rather than branch management. However, keep in mind that you may set or encounter different branching rules in future Git projects that you work on. It's important to communicate with your collaborators about branching conventions and to have a good understanding of what branches are and how working in local and remote repositories with other people affects branches.

Now, let's walk through some examples to see how the collaboration on the Rainbow project works.

MAKING A COMMIT IN THE LOCAL REPOSITORY

In this section, your friend is going to add a note about the color green to the rainbowcolors.txt file and make a commit on the main branch of their local repository. Remember that you're acting as your friend now, which means you'll be working in their project directory, friend-rainbow. As discussed earlier, you should have this project directory open in a separate text editor window.

Go to Follow Along 8-5 to simulate your friend making a commit.

[NOTE]

In the git log output of step 4 in Follow Along 8-5, you are the author of both the yellow commit and the green commit. This is because you are simulating that you are your friend contributing to the Rainbow project. However, in reality, if someone else had made the green commit, then they would be the author of the commit, and their name and email address would be displayed in the output. Keep this in mind for the rest of the chapters in this book.

[FOLLOW ALONG 8-5]

1. Open the rainbowcolors.txt file in the friend-rainbow project directory in your text editor, add "Green is the fourth color of the rainbow." on line 4, and save the file.

2. friend-rainbow $ **git add rainbowcolors.txt**

3. friend-rainbow $ **git commit -m "green"**
   [main 6987cd2] green
   1 file changed, 2 insertions(+), 1 deletion(-)

4. friend-rainbow $ **git log**
   commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (**HEAD -> main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 11:49:03 2022 +0100
   green
   commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**origin/main, origin/HEAD**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 10:09:59 2022 +0100
   yellow

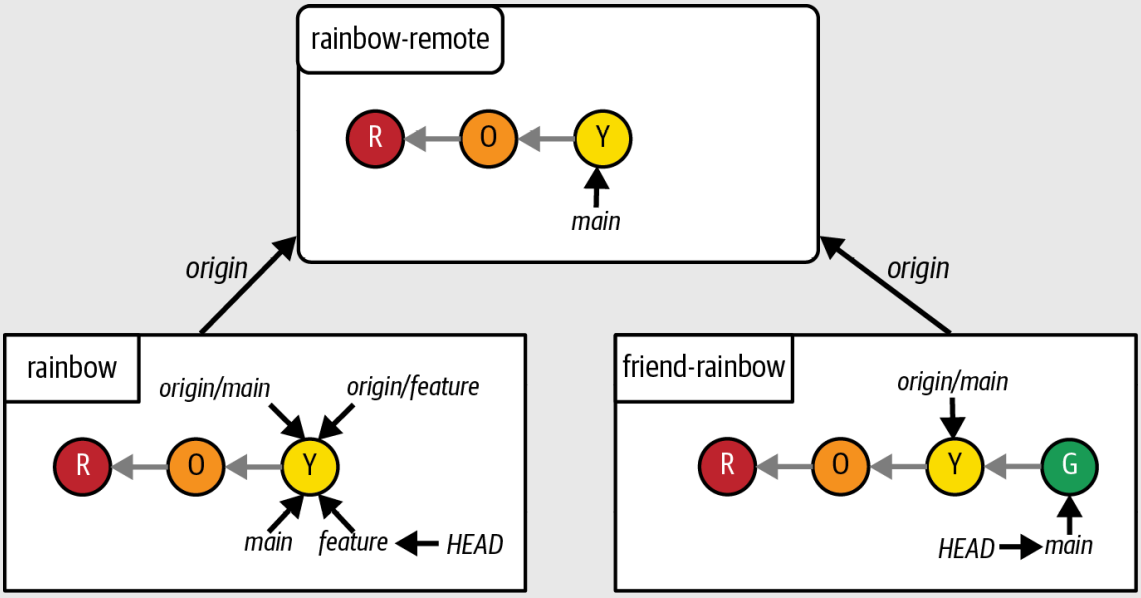
5. Go to the rainbow-remote repository on your hosting service and refresh the page. The green commit won't be there yet.

What to notice:

- In step 4, the git log output indicates that in the friend-rainbow repository:
  - The local main branch has updated to point to the green commit.
  - The origin/main remote-tracking branch still points to the yellow commit.

Visualize it 8-5 illustrates the changes in the friend-rainbow repository.

[VISUALIZE IT 8-5]



The Rainbow project after your friend makes the green commit on the main branch in the friend-rainbow repository

As you can see, the green commit does not yet appear in the rainbow-remote repository. This is because remote repositories do not update automatically; work done in local repositories needs to be explicitly pushed to them.

Notice also that origin/main remote-tracking branch still points to the yellow commit. This is because, as mentioned in Chapter 7, remote-tracking branches represent the state of remote branches. Since your friend has not yet pushed their changes to the remote repository, the remote main branch has not updated, and therefore the origin/main remote-tracking branch has not updated either.

At this point there is no way for you to have the green commit in the rainbow repository, because updating one local repository with the changes made in another local repository

is a two-step process. First, the local repository with the changes has to explicitly push those changes to a remote repository. Then, the local repository without the changes has to explicitly fetch and integrate the changes from the remote repository.

Next, let's see how your friend will push the work they've done to the remote repository.

PUSHING TO THE REMOTE REPOSITORY

In Chapter 7, you learned that when you push work from a local branch to a remote branch, Git needs some way to know which remote branch you want to push to. If the local branch has an upstream branch defined for it, you can use git push with no arguments and Git will automatically push the work to that branch. However, if no upstream branch is defined for the local branch you're working on, you'll need to specify which remote branch to push to when you enter the git push command.

The last time you used the git push command in the rainbow repository, you passed in the shortname and the branch name. This was because you had not defined an upstream branch for your local branch. Recall that an upstream branch is the remote branch that a particular local branch tracks. When you clone a repository, upstream branches are *automatically* set up for the branches that exist in the cloned repository.

The command to use to see whether upstream branches are defined is git branch with the -vv option (which stands for "very verbose"). This command also tells you whether a local branch is ahead of or behind an upstream branch, if it is defined.

[SAVE THE COMMAND]

git branch -vv

List the local branches and their upstream branches, if they have any

There are two main benefits to defining upstream branches:

1. If you have fetched (downloaded) the remote repository commits, you can check if your repository is ahead of or behind the remote repository by using either the git branch -vv command or the git status command. (We will cover the process of fetching in the next section in this chapter.)

2. You can simplify the commands you use with the remote repository because you don't need to specify the branch name and remote repository shortname. For example, you can use the git push command on its own without passing any arguments.

We will observe both of these benefits in the upcoming Follow Alongs.

Since your friend cloned their repository, the upstream branch for their local main branch is already set to be the remote main branch. Go to Follow Along 8-6 to see how your friend can check the status of the remote repository.

[FOLLOW ALONG 8-6]

1. friend-rainbow $ **git branch -vv**
   * main 6987cd2 [origin/main: ahead 1] green

2. friend-rainbow $ **git status**
   On branch main
   Your branch is ahead of 'origin/main' by 1 commit.
   (use "git push" to publish your local commits)
   nothing to commit, working tree clean

What to notice:

- In step 1, the git branch -vv output shows that the upstream branch set up for the local main branch is the remote main branch in the remote repository with shortname origin and that the local main branch is ahead by one commit.
- In step 2, the git status output also indicates that the local main branch is ahead of the origin/main upstream branch by one commit.

Next, go to Follow Along 8-7 to see how your friend can simply use the git push command without any arguments to update the remote repository.

[FOLLOW ALONG 8-7]

1. friend-rainbow $ **git push**

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 295 bytes | 295.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:gitlearningjourney/rainbow-remote.git
fc8139c..6987cd2 main -> main
```

2. friend-rainbow $ **git status**

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

3. friend-rainbow $ **git log**

```
commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (HEAD -> main, origin/main, origin/HEAD)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green
commit fc8139cbf8442cdbb5e469285abaac6de919ace6
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
```

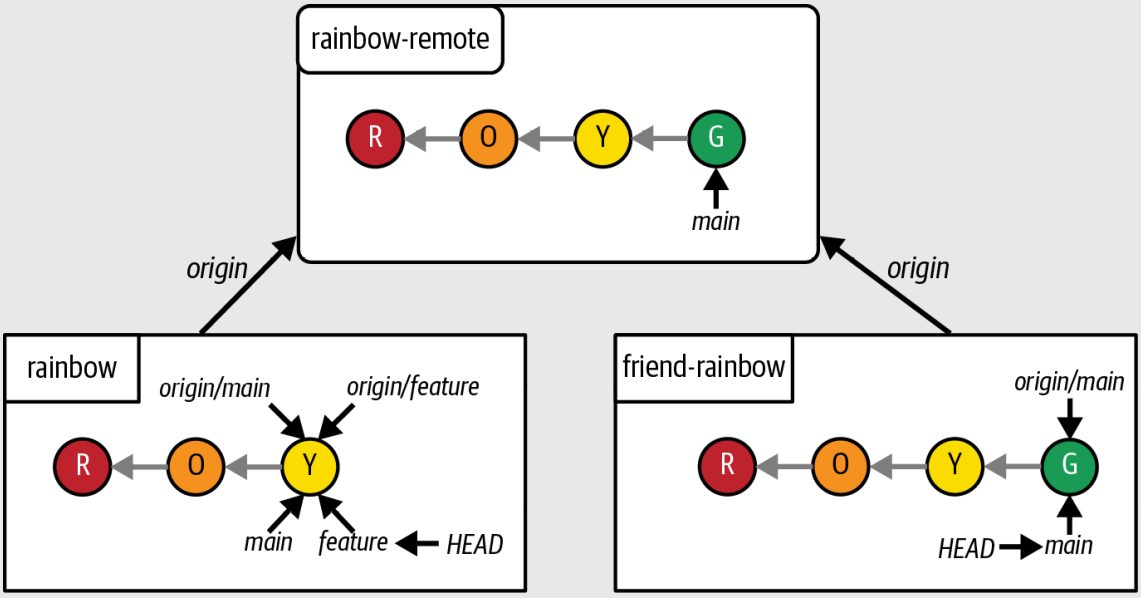
4. Go to the rainbow-remote repository on your hosting service and refresh the page. The green commit should now be there.

What to notice:

- In step 3, the git log output indicates that in the friend-rainbow repository, the origin/main remote-tracking branch has updated to point to the green commit.
- In step 4, you can see that in the rainbow-remote repository, the remote main branch has been updated to point to the green commit.

Visualize it 8-6 illustrates these observations.

[VISUALIZE IT 8-6]



The Rainbow project after your friend pushes the green commit to the remote repository

Notice that the rainbow repository does not yet have the green commit, and that the local main branch and origin/main remote-tracking branch in this repository still point to the yellow commit.

Next, you want to make sure that the local main branch in your rainbow repository is in sync with the main branch in the rainbow-remote repository and the friend-rainbow repository. For that, you'll need to learn about fetching.

Incorporating Changes from the Remote Repository

The reason the local main branch and origin/main remote-tracking branch in the rainbow repository still point to the yellow commit is because local repositories do not automatically update with new data from remote repositories. Just as you need to take explicit actions to update a remote repository with changes in a local repository, you need to

take explicit actions to update local branches and remote-tracking branches with changes from a remote repository.

Incorporating changes from a remote branch into a local branch is a two-step process: first you fetch the changes from the remote repository, then you integrate those changes into the local branch in the local repository. Let's start by exploring the first step.

FETCHING CHANGES FROM THE REMOTE REPOSITORY

In Git, we use the term *fetch* or *fetching* to refer to the process of downloading data from a remote repository to a local repository, and the command we use to do this is git fetch. The git fetch command downloads all the necessary commits to update all the remote-tracking branches in the local repository to reflect the state of the remote branches in the remote repository that is specified. When no remote repository shortname is provided as an argument to the git fetch command, by default the remote repository with shortname origin will be used, unless there's an upstream branch defined for the current branch.

[SAVE THE COMMAND]

git fetch <shortname>

Download data from the <shortname> remote repository

git fetch

Download data from the remote repository with shortname origin

The git fetch command affects only remote-tracking branches. It does not affect local branches. In other words, it only fetches (downloads) data; it doesn't actually integrate the data into any local branches. Thus, nothing in your working directory will change when you fetch data from a remote repository.

You'll integrate the changes in the next section; for now, go to Follow Along 8-8 to fetch the data from the remote repository.

[FOLLOW ALONG 8-8]

1. Go to the rainbow project directory in your command line window.

2. rainbow $ **git log --all**

```
commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (HEAD -> feature, origin/main, origin/feature, main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
commit 7acb333f08e12020efb5c6b563b285040c9dba93
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange
```

3. rainbow $ **git fetch**

```
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0
Unpacking objects: 100% (3/3), 275 bytes | 91.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
fc8139c..6987cd2 main -> origin/main
```

4. rainbow $ **git log --all**

```
commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (origin/main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green
commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (HEAD -> feature, origin/feature, main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
```

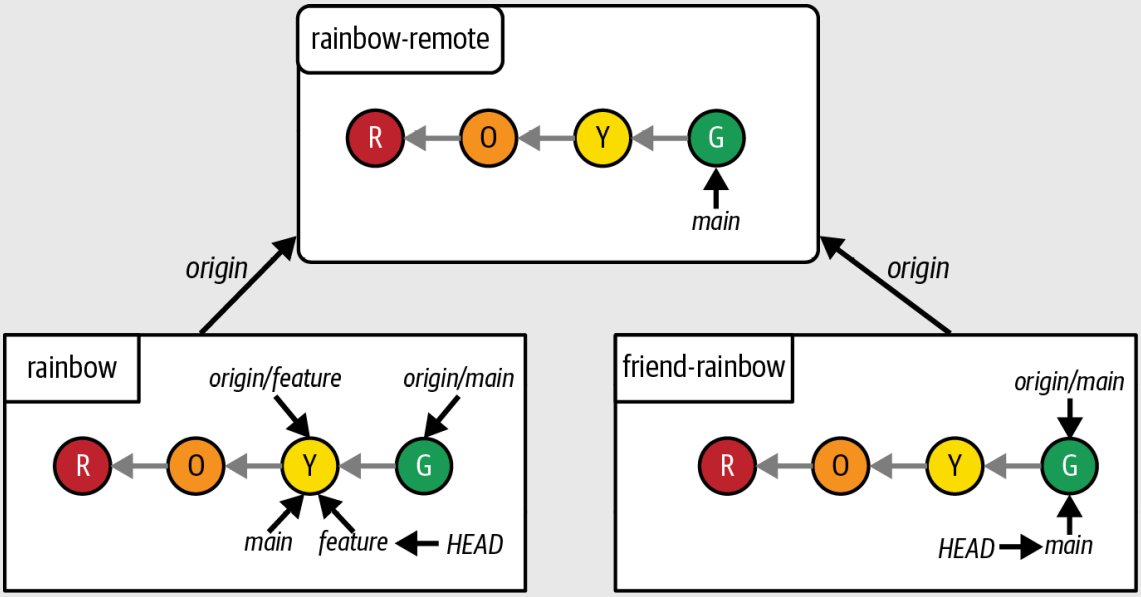
What to notice:

- In step 2, the git log output shows that:

    - The local main branch and the origin/main remote-tracking branch are pointing to the yellow commit.
- In step 4, the git log output shows that:
    - The origin/main remote-tracking branch is pointing to the green commit.
    - The local main branch is still pointing to the yellow commit.

Visualize it 8-7 illustrates these observations.

[VISUALIZE IT 8-7]



The Rainbow project after you fetch the data from the rainbow-remote repository to the rainbow repository

You just saw how the git fetch command updated the remote-tracking branches in the local repository for any remote branches that exist in the remote repository. Now, you are ready for the second step in the process: incorporating the changes from the remote repository into a local branch.

INTEGRATING CHANGES INTO A LOCAL BRANCH

Once you have fetched the changes from a remote repository and updated the remote-tracking branches in a local repository, you're ready to update a local branch. Recall that in Chapter 5, I mentioned that Git provides two ways to integrate changes: *merging* and

rebasing. We will cover rebasing in Chapter 11. For now, you will continue using merging.

In "Types of Merges" on page 64, you learned about two kinds of merges: fast-forward merges and three-way merges. In the fast-forward merge you carried out in Chapter 5, you merged a branch called feature into main. Both feature and main were local branches in the rainbow repository. In this section, you are going to merge the origin/main remote-tracking branch into the local main branch in the rainbow repository. The types of branches involved in the merge are different, but the process of executing the merge is the same. This will once again be a fast-forward merge.

As you learned in Chapter 5, when you execute a merge you must be on the branch you're merging into, which in this case will be the main branch in the rainbow repository. Therefore, in Follow Along 8-9, you will first switch onto the main branch before executing the merge.

To integrate the commits that were on the main branch in the remote repository into your local main branch, you will use the git merge command. This time you will pass in the name of a remote-tracking branch, origin/main.

Go to Follow Along 8-9 to execute the merge.

[FOLLOW ALONG 8-9]

1. rainbow $ **git switch main**
   Switched to branch 'main'

2. rainbow $ **git merge origin/main**
   Updating fc8139c..6987cd2
   Fast-forward
   rainbowcolors.txt | 3 ++-
   1 file changed, 2 insertions(+), 1 deletion(-)

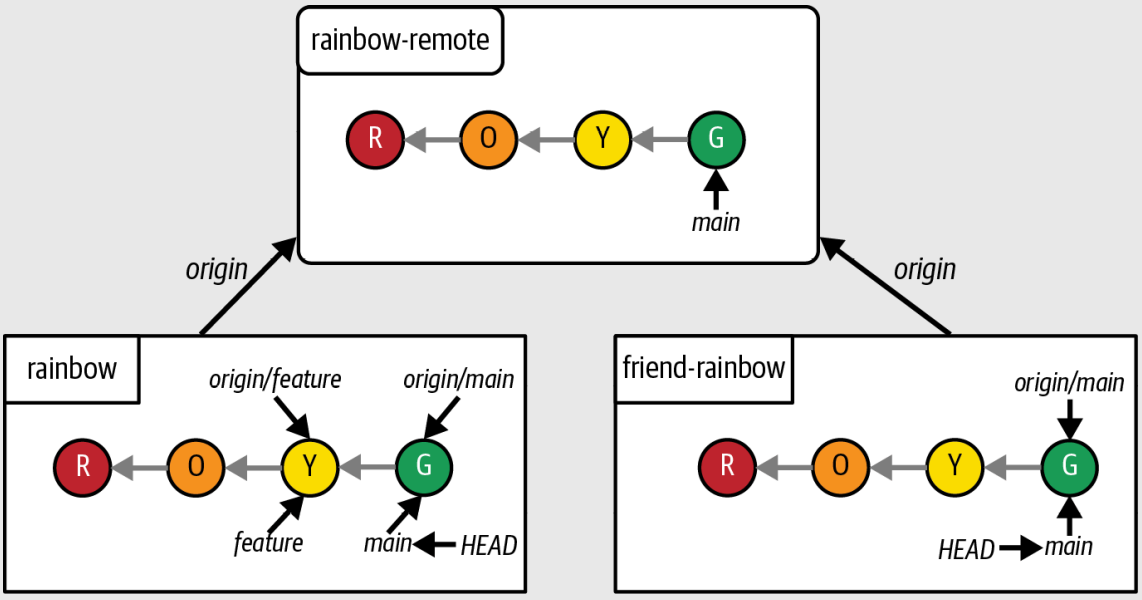
3. rainbow $ **git log**
   commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (**HEAD -> main**, **origin/main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 11:49:03 2022 +0100
   green
   commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**origin/feature**, **feature**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 10:09:59 2022 +0100
   yellow

What to notice:

- In step 3, the git log output indicates that the local main branch points to the green commit.

This is illustrated in Visualize it 8-8.

[VISUALIZE IT 8-8]



The Rainbow project after you merge the origin/main remote-tracking branch into the local main branch in the rainbow repository

You have just observed how the local branch updates once you complete the merge. In Visualize it 8-8, you can also see that in the rainbow repository you still have the feature branch. Next, let's carry out some more branch deletions to declutter the rainbow repository.

Deleting Branches (Continued)

In the rainbow repository, you still have the local feature branch and the origin/feature remote-tracking branch. For simplicity, moving forward you will work on the main branch, so you can now delete those branches.

In "Deleting Branches" on page 122, you used the git push <shortname> -d <branch_name> command to delete the remote feature branch and the origin/feature remote-tracking branch in the friend-rainbow repository in one go. Now there is no remote feature branch any more, so to delete the origin/feature remote-tracking branch in the rainbow repository you can use the git fetch command with the -p option (which stands for "prune"). This command with this option will delete any remote-tracking branches that correspond to remote branches that have been deleted in the remote repository.

[SAVE THE COMMAND]

git fetch -p

> Remove remote-tracking branches that correspond to deleted remote branches and download data from the remote repository

To delete the local feature branch, you will use the same command you used earlier in this chapter. Go to Follow Along 8-10 to carry out the branch deletions.

[FOLLOW ALONG 8-10]

1. rainbow $ **git branch --all**
   feature
   * main
   remotes/origin/feature
   remotes/origin/main

2. rainbow $ **git fetch -p**
   From github.com:gitlearningjourney/rainbow-remote
   - [deleted] (none) -> origin/feature

3. rainbow $ **git branch --all**
   feature
   * main
   remotes/origin/main

4. rainbow $ **git branch -d feature**
   Deleted branch feature (was fc8139c).

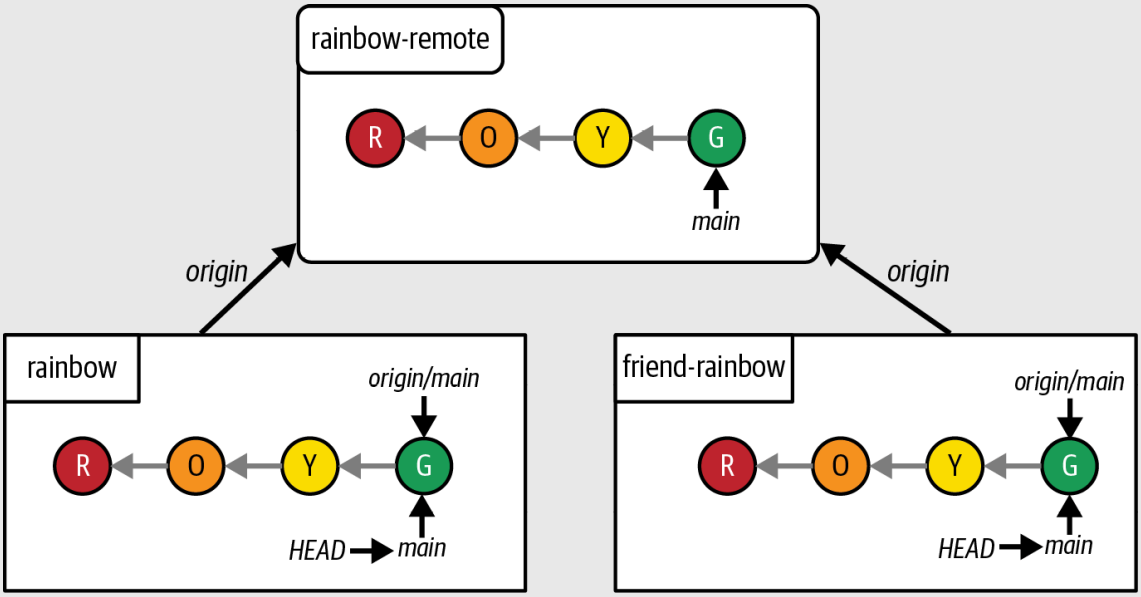
5. rainbow $ **git branch --all**
   * main
   remotes/origin/main

What to notice:

- In step 2, you deleted the origin/feature remote-tracking branch.
- In step 4, you deleted the local feature branch.
- In step 5, the git branch --all output indicates that you no longer have a local feature branch or an origin/feature remote-tracking branch in the rainbow repository.

This is illustrated in Visualize it 8-9.

[VISUALIZE IT 8-9]



The Rainbow project after you delete the local feature branch and the origin/feature remote-tracking branch in the rainbow repository

Summary

In this chapter, you started simulating what it would be like to work with other people on a Git project. Your "friend" cloned the rainbow-remote repository and created a second local repository, which you called friend-rainbow. Through this process, you observed that origin is the default shortname that Git associates with a remote repository when it is cloned.

After your friend made the green commit in the friend-rainbow repository and pushed it to the remote repository, you learned about the process of updating a local branch with

changes from a remote branch in a remote repository, which involves fetching the changes from the remote repository and then integrating them. In the rainbow repository, you integrated the changes by carrying out a fast-forward merge. Next, in Chapter 9, you will learn more about the other kind of merge mentioned in Chapter 5: the three-way merge.

Previous chapter
< Chapter 7. Creating and Pushing to a Remote Repository

Next chapter
Chapter 9. Three-Way Merges >